ASYNC AWAIT IN PYTHON

-By Anushka Raj

Introduction

“WHY DOES MULTITHREADING IN PYTHON NOT FULLY USE CPU?”

“EVEN IF WE CREATE MULTIPLE THREADS IN PYTHON, ONLY ONE RUNS AT A TIME... WHY?”

GIL

The **GIL (Global Interpreter Lock)** is a concept in Python that affects how programs run, especially when using threads.

Due to Global Interpreter Lock (GIL), Python cannot provide parallelism similar to what multithreading works in other languages, such as Java/C++

Why GIL?

- Core Reason: Memory Safety
- Python uses a system called reference counting for memory management.
- Every object keeps track of how many references point to it.
- When the count becomes 0 – memory is freed.

Without GIL

If multiple threads modify the reference count at the same time:

- Count may become incorrect
- Objects may be deleted too early or never deleted
- Leads to crashes or memory corruption

GIL ensures:

- Only one thread updates memory at a time – safe execution

Does GIL prevent concurrency?

While it limits parallel execution of threads, there are ways to achieve concurrency in Python-

- **MULTITHREADING -** good for I/O tasks
- **MULTI-PROCESSING -** true parallelism (CPU task)
- **ASYNCIO (async/await) -** concurrency without threads

MULTI-THREADING

- **Multithreading** in Python means running multiple threads (smaller units of a program) at the same time within a single process.
- Python rapidly switches between threads. This switching is so fast that it looks parallel, but actually it is not. Due to GIL, only one thread executes Python bytecode at a time.
- For example: while waiting in line for billing, someone start scrolling reels.
  Here, billing is one thread and scrolling reels is another.
  Python threads utilize waiting time by switching between tasks, achieving concurrency.
- Multi-threading works well for I/O bound tasks, but it is limited for CPU-bound tasks because of the GIL.

Example use cases of Multi Threading:

- Multiple download files:
    - Each file download runs in a separate thread
    - While one file is waiting for network response, others continue downloading
    - Much faster than downloading one by one

- Web Scraping / API Calls(I/O bound tasks)
    - Sending multiple requests simultaneously
    - Threads don’t wait idle – better performance

Without Threading

```python
import time

start = time.time()

def print_numbers():
    for i in range(5):
        print(f"Numbers: {i}")
        time.sleep(1)

def print_letters():
    for letter in ['A','B','C','D','E']:
        print(f"Letter: {letter}")
        time.sleep(1)

print_numbers()
print_letters()

end = time.time()

print("Time taken:", end-start)
print("Done")
```

```
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python> python -u MultiThreading.py
Numbers: 0
Numbers: 1
Numbers: 2
Numbers: 3
Numbers: 4
Letter: A
Letter: B
Letter: C
Letter: D
Letter: E
Time taken: 10.009031534194946
Done
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python>
```

With Threading

```python
import threading
import time

start = time.time()

def print_numbers():
    for i in range(5):
        print(f"Number: {i}")
        time.sleep(1)

def print_letters():
    for letter in ['A', 'B', 'C', 'D', 'E']:
        print(f"Letter: {letter}")
        time.sleep(1)

t1 = threading.Thread(target=print_numbers)
t2 = threading.Thread(target=print_letters)

t1.start()
t2.start()

t1.join()
t2.join()

end = time.time()
print("Time taken:", end-start)
print("Done")
```

```
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python> python -u MultiThreading.py
Number: 0
Letter: A
Number: 1
Letter: B
Number: 2
Letter: C
Number: 3
Letter: D
Number: 4
Letter: E
Time taken: 5.013087272644043
Done
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python>
```

Here t1 is one thread and t2 is another.
While t1 is waiting t2 starts running and when t2 is waiting t1 starts running.

MULTI-PROCESSING

- **Multiprocessing** in Python means running multiple processes at the same time, where each process has its own memory space and Python interpreter.
- Each process has its own GIL, so multiple CPU cores can be used simultaneously.

Key Features:

- True parallel execution
- Separate memory (no shared variables by default)
- Avoids GIL limitations
- Slightly heavier than threading (more memory + startup time)

Example use cases of Multi Processing:

- Image Processing:
    - Applying filters/resizing hundreds of images
- Mathematical Computation
    - Heavy math operations
- Parallel Data Processing
- Web Backend
    - When API does heavy computation

Without Multi_Processing

```python
import time

start = time.time()

def square(n):
    for i in range(5):
        print(f"{n} squared = {n*n}")
        time.sleep(1)

for i in range(3):
    square(i)

end = time.time()
print("Time taken:", end-start)
```

```
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python> python -u Multi_Processing.py
0 squared = 0
0 squared = 0
0 squared = 0
0 squared = 0
0 squared = 0
1 squared = 1
1 squared = 1
1 squared = 1
1 squared = 1
1 squared = 1
2 squared = 4
2 squared = 4
2 squared = 4
2 squared = 4
2 squared = 4
Time taken: 15.024989604949951
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python>
```

With Multi_Processing

```python
from multiprocessing import Process
import time

def square(n):
    for i in range(5):
        print(f"{n} squared = {n*n}")
        time.sleep(1)

if __name__ == "__main__":
    processes = []
    start = time.time()

    for i in range(3):
        p = Process(target=square, args=(i,))
        processes.append(p)
        p.start()

    for p in processes:
        p.join()

    end = time.time()
    print("Time:", end - start)
```

```
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python> python -u Multi_Processing.py
0 squared = 0
1 squared = 1
2 squared = 4
0 squared = 0
1 squared = 1
2 squared = 4
0 squared = 0
1 squared = 1
2 squared = 4
0 squared = 0
1 squared = 1
2 squared = 4
0 squared = 0
1 squared = 1
2 squared = 4
Time: 5.160454750061035
```

- For different i there are different processes.
- Each process have different CPU cores and their own GIL.
- And they are running parallely.

ASYNCIO (async/await)

- **Asyncio** in Python is a library used to write asynchronous (non-blocking) code using a single thread. It's mainly used when you have a lot of waiting tasks (like network requests, file I/O, APIs, etc.).
- Concurrency (Not Parallelism)
- Asyncio does NOT use multiple cores.
- It uses smart scheduling in one thread.
- It works using an event loop that schedules tasks when they are waiting (await).
- Good for:
  - Network requests
  - APIs
  - Web scraping
  - Chat servers

Example use cases of Async/Await:

- Making Multiple API Calls
    - Fetching data from multiple APIs simultaneously

- Chat Applications / Messaging Systems
    - Handling multiple users at once

- Downloading Multiple Files Efficiently

Without using Asyncio

```python
import time

start = time.time()

def task(name):
    print(f"Task {name} started")
    time.sleep(2)
    print(f"Task {name} finished")

t1 = task("A")
t2 = task("B")

end = time.time()
print("Time taken: ", end-start)
```

```
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python> python -u Asyncio.py
Task A started
Task A finished
Task B started
Task B finished
Time taken:  4.004236698150635
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python>
```

With using Asyncio

```python
import asyncio
import time

start = time.time()
async def task(name):
    print(f"Task {name} started")
    await asyncio.sleep(2)
    print(f"Task {name} finished")

async def main():
    t1 = asyncio.create_task(task("A"))
    t2 = asyncio.create_task(task("B"))

    await t1
    await t2

asyncio.run(main())
end = time.time()
print("Time taken: ",end-start)
```

```
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python> python -u Asyncio.py
Task A started
Task B started
Task A finished
Task B finished
Time taken:  2.023188829421997
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python>
```

Here t1 and t2 are in one thread.
When t1 is awaited t2 runs and when t2 is awaited t1 runs.

HTTP Request Synchronously

```python
import requests
import time

start_time = time.time()


for i in range(1,151):
    url = 'https://pokeapi.co/api/v2/pokemon/' + str(i)
    resp = requests.get(url)
    pokemon = resp.json()
    print(pokemon['name'])

end_time = time.time()

print('Execution time',(end_time-start_time))
```

```
articuno
zapdos
moltres
dratini
dragonair
dragonite
mewtwo
Execution time 152.27775764465332
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python>
```

- Synchronous API call
- Getting response for first call and then requesting other

HTTP Request Asynchronously

```python
import aiohttp
import asyncio
import time

start_time = time.time()

async def main():
    async with aiohttp.ClientSession() as session:
        for i in range(1,151):
            url = 'https://pokeapi.co/api/v2/pokemon/' + str(i)
            async with session.get(url) as resp:
                pokemon = await resp.json()
                print(pokemon['name'])


asyncio.run(main())
end_time = time.time()
print('Execution time',(end_time-start_time))
```

```
snorlax
articuno
articuno
zapdos
moltres
dratini
dragonair
dragonite
mewtwo
Execution time 27.24150776863098
PS C:\Users\Anushka Raj\OneDrive\Desktop\coding\python>
```

- Asynchronous API call using aiohttp
- Calling all requests together

Multithreading and async/await may look similar, but they are fundamentally different.

MULTI_THREADING

- Multi Threads
- OS decides which thread runs (context switching)
- Used for:
  - Multiple I/O tasks
  - Blocking operations

ASYNC/AWAIT

- Single Threads
- We decide which task is to be awaited and when.
- Uses an event loop.
- Used for:
  - Network calls(APIs)
  - Web scraping
  - Chat servers

CONCLUSION

- Due to GIL, multithreading does not achieve true parallelism but there are ways to achieve concurrency.
- MultiThreading executes multiple threads efficiently.
- Multiprocessing enables true parallel execution using multiple CPU cores.
- Async/Await provides concurrency using a single thread and an event loop.

THANK YOU